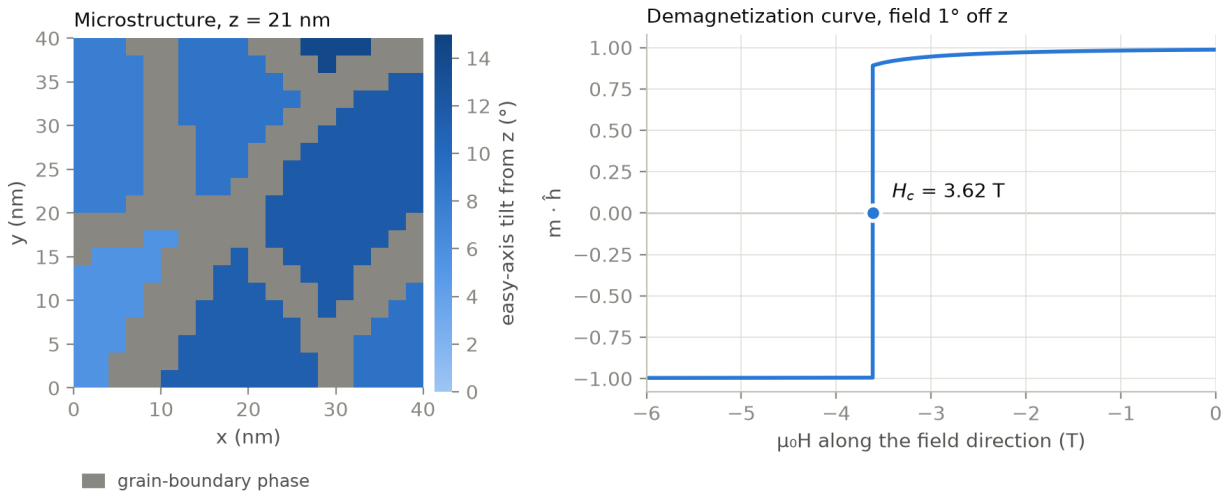
micromag

A user's guide to the finite-difference micromagnetic solver

version 0.0.1 · github.com/tanooj-s/micromag · GPL-3.0-or-later



An 8-grain $Nd_2Fe_{14}B$ polycrystal with a soft grain-boundary phase (left) and its demagnetisation curve (right), run from `examples/grains.toml` in about a minute on a laptop CPU.

Contents

1 What micromag is **2**

2 Installing and running **3**

3 The physical model **3**

4 Implementation choices **4**

4.1 Grid and data layout . . . 4
4.2 Exchange . . . 4
4.3 Demagnetising field . . . 4
4.4 Periodic boundaries . . . 4
4.5 Time integration . . . 5
4.6 Energy minimisation and hysteresis . . . 5
4.7 Energy barriers . . . 5
4.8 Backends . . . 5

5 Kinds of simulations **5**

6 Input card reference **7**

6.1 Top level and `[grid]` . . . 7
6.2 `[material.<name>]` . . . 7
6.3 `[[region]]`: painted shapes . . . 7

6.4 [grains]: Voronoi polycrystals 8
6.5 [[coupling]]: exchange between regions 8
6.6 [initial] 8
6.7 [[stage]], type = "minimize" 9
6.8 [[stage]], type = "hysteresis" 9
6.9 [[stage]], type = "gneb" 9
6.10 [[stage]], type = "llg-dynamics" 9

7 Choosing numerical parameters **10**

8 Thermal runs: what to expect **10**

9 The results folder **11**

10 The Python API **11**

11 Validation and known limits **12**

1 What micromag is

micromag computes how the magnetisation inside a small magnet, from a few nanometres to a few hundred, arranges itself and moves. It divides the magnet into a regular grid of box-shaped cells, gives each cell a magnetisation direction **m** (a unit vector), and then either lets **m** evolve in time or searches for the arrangement of lowest energy.

It is written from scratch in C++, with an optional CUDA backend, and driven from Python. It follows the micromagnetic model of the review by Fischbacher *et al.* (2018) [1], and its scope is modelled on the micromagnetic part of MagTense. A simulation is described by one TOML file, the *input card*, and produces one results folder.

What it can do.

- Four energy terms: exchange, uniaxial anisotropy, an applied (Zeeman) field and the magnetostatic (demagnetising) field, the last computed exactly from the Newell tensor by FFT.
- Three kinds of calculation: time evolution by the Landau–Lifshitz–Gilbert (LLG) equation, energy minimisation, and quasi-static hysteresis sweeps with automatic refinement near switching.
- A thermal field (Brown's stochastic field) for finite-temperature dynamics.
- Materials that vary cell by cell: named materials painted onto the grid as shapes, Voronoi polycrystals with per-grain easy axes and an optional grain-boundary phase, and exchange coupling set per pair of regions.
- Free or periodic boundaries on any axis, so a small tile can stand in for an extended film, a wire or bulk material.
- The same card on a CPU or an NVIDIA GPU, in double precision, with the same results on both.

What it does not do (yet). Cubic or higher-order anisotropy, the Dzyaloshinskii–Moriya interaction, magnetoelastic coupling, spatially varying applied fields or temperatures, and multi-GPU runs are not implemented. For production research, the mature codes are mumax3, OOMMF, MagTense and Ubermag; micromag is a young code validated against the standard benchmarks (Section 11).

Conventions. SI units throughout: A/m, J/m, J/m^3, m, s, K. Applied fields in the card are given as $\mu_0\mathbf{H}$ in *tesla*, because the literature quotes T and mT. Damping is the *dimensionless Gilbert* α. Each design choice has a numbered entry D1, D2, . . . in `DECISIONS.md`, with the alternatives that were rejected; this guide cites them in grey, e.g. (D11).

2 Installing and running

micromag builds from source with uv. You need a C++17 compiler; uv fetches Python (≥ 3.11), CMake and the Python dependencies.

```
git clone https://github.com/tanooj-s/micromag.git
cd micromag
uv sync                                      # CPU build
uv run micromag run examples/grains.toml -o runs/grains
```

The run prints one progress line per stage and writes `runs/grains/`. The output folder must be new or empty, so a result is never overwritten.

GPU build. Install the CUDA toolkit (12.x or 13.x) with `nvcc` on `PATH`, then

```
export SKBUILD_CMAKE_DEFINE="MICROMAG_CUDA=ON"
uv sync
uv run python -c "import micromag; print(micromag.has_cuda)"     # True
uv run micromag run examples/sp2.toml -o runs/sp2 --device cuda
```

Keep `SKBUILD_CMAKE_DEFINE` set in every later shell: `uv run` rebuilds when it changes, so forgetting it silently rebuilds a CPU-only extension. Any NVIDIA GPU of compute capability 7.0 or newer works with CUDA 12 (7.5 with CUDA 13). Because everything is float64, data-centre cards (V100, A100, H100) are much faster than consumer cards or the T4. `docs/gpu.md` covers Google Colab and Slurm clusters.

Apple Silicon. If the `cmake` on `PATH` is an x86_64 binary, `export ARCHFLAGS="-arch arm64"` first, or the extension builds for the wrong architecture.

Tests. `uv run pytest` runs the fast suite (about a minute). `uv run pytest -m slow -s` runs the physics benchmarks (about 15 minutes on a laptop CPU) and prints the comparisons.

3 The physical model

The state is the unit vector field $\mathbf{m}(\mathbf{r}) = \mathbf{M}/M_s$. Each cell carries its own saturation magnetisation M_s, exchange stiffness A, anisotropy constant K and easy axis $\hat{\mathbf{k}}$. The total energy is the sum of four terms, and each has an effective field $\mathbf{H} = -\frac{1}{\mu_0 M_s}\,\delta E/\delta \mathbf{m}$ (Table 1).

Table 1: The energy terms. Equation numbers refer to Fischbacher *et al.* (2018).

Term	Eq.	Energy density	Field $\mathbf{H}$	Parameters
Exchange	8	$A\,\lvert\nabla \mathbf{m}\rvert^2$	$\frac{2}{\mu_0 M_s}\nabla\cdot(A\nabla\mathbf{m})$	A (J/m)
Anisotropy	9	$-K(\mathbf{m}\cdot\hat{\mathbf{k}})^2$	$\frac{2K}{\mu_0 M_s}(\mathbf{m}\cdot\hat{\mathbf{k}})\,\hat{\mathbf{k}}$	K (J/m^3), $\hat{\mathbf{k}}$
Zeeman	11	$-\mu_0 M_s\,\mathbf{m}\cdot\mathbf{H}_{\text{ext}}$	$\mathbf{H}_{\text{ext}}$	$\mu_0\mathbf{H}_{\text{ext}}$ (T), per stage
Demag	17–21	$-\frac{1}{2}\mu_0 M_s\,\mathbf{m}\cdot\mathbf{H}_d$	$\mathbf{H}_d = -\mathcal{N} * (M_s\mathbf{m})$	none (geometry)

The anisotropy uses the paper's sign: a uniform $\mathbf{m} \parallel \hat{\mathbf{k}}$ has energy $-KV$ (D12). $K > 0$ gives an easy axis, $K < 0$ an easy plane. Vacuum is simply $M_s = 0$: such cells have no field, no energy and no exchange neighbours (D11).

Dynamics. The LLG equation (eq. 25) is integrated in its Landau–Lifshitz form,

$$\frac{d\mathbf{m}}{dt} = -\gamma'\,\mathbf{m}\times\mathbf{H}_{\text{eff}} - \alpha\gamma'\,\mathbf{m}\times(\mathbf{m}\times\mathbf{H}_{\text{eff}}), \qquad \gamma' = \frac{\mu_0|\gamma|}{1+\alpha^2},$$

with $\mathbf{H}_{\text{eff}}$ the sum of the four fields (plus the thermal field at $T > 0$). The first term makes $\mathbf{m}$ precess about the field; the second, the damping, turns it towards the field. γ defaults to 1.76086×10^{11} rad/(T s).

Equilibrium. For slowly varying fields, as in a permanent magnet's hysteresis loop, the magnetisation sits in a local energy minimum at each field (paper §2.2.3). Equilibrium means zero torque, $\mathbf{m}\times\mathbf{H}_{\text{eff}} = 0$, and every

stopping test in micromag is on the largest per-cell torque $\max|\mathbf{m}\times\mathbf{H}_{\text{eff}}|/M_s$.

Temperature. Brown's model adds a Gaussian white-noise field $\mathbf{H}_{\text{th}}$ to $\mathbf{H}_{\text{eff}}$, independent in every cell, component and step, with variance fixed by the fluctuation–dissipation theorem (D41):

$$\sigma^2 = \frac{2\,\alpha\,k_B T}{\mu_0\,\gamma_0\,M_s\,V\,\Delta t}, \qquad \gamma_0 = \mu_0|\gamma|,$$

where V is the cell volume and Δt the step. Larger damping needs stronger noise; a larger moment $M_s V$ needs less. There is no $1+\alpha^2$ factor: the noise enters $\mathbf{H}$ in Gilbert's form, as in mumax3.

4 Implementation choices

This section explains the numerical choices a user should know about, because they set accuracy, cost and the limits of what can be simulated.

4.1 Grid and data layout

The grid is a box of $n_x \times n_y \times n_z$ cells of size $\Delta x \times \Delta y \times \Delta z$. Coordinates in the card are measured in metres from the lower corner; cell (i, j, k) is centred at $((i+\frac{1}{2})\Delta x, (j+\frac{1}{2})\Delta y, (k+\frac{1}{2})\Delta z)$. All arrays are float64 *structure-of-arrays*: vectors are shaped `(3, nz, ny, nx)` and scalars `(nz, ny, nx)`, with x fastest (D9). This layout feeds the FFTs and the GPU directly. Every material parameter is stored per cell, so the solver core never knows about shapes or grains: geometry is expanded into per-cell arrays in Python, and new geometry never touches C++.

4.2 Exchange

A second-order six-neighbour Laplacian (D11). The coupling across a face between two cells uses the *harmonic mean* $A_{\text{face}} = 2A_1A_2/(A_1+A_2)$, which is zero if either side is zero, so a hard grain cannot couple straight through a non-magnetic boundary. The surfaces are free (Neumann): a missing or vacuum neighbour contributes nothing. Across faces between two different regions, A_{face} can be scaled or replaced by a `[[coupling]]` entry (D35), which models thin, weak interfaces without resolving them with cells. On a periodic axis the neighbour across the boundary is the cell on the opposite face (D45).

4.3 Demagnetising field

The demagnetising field is a convolution of the magnetisation with the demagnetisation tensor $\mathcal{N}$, which depends only on the offset between two cells (D14, D15).

- **Tensor.** The exact Newell formulas for cell pairs closer than 20 cells; beyond that, a point-dipole kernel averaged over $3\times3\times3$ Gauss points in each cell, because Newell's formulas lose precision far away (they sum 27 large, nearly cancelling terms). The tensor is accurate to about 10^{-6} relative, far below the $O(\Delta x^2)$ discretisation error. Cells flatter than about 1:4 are untested.
- **Convolution.** Zero-padded to $2n$ per free axis (an axis with $n = 1$ stays at 1) and done with real-to-complex FFTs: pocketfft on the CPU, cuFFT on the GPU. The tensor's spectrum is computed once per run.
- **Cost.** The padding makes demag the dominant cost in memory and time: about 8× the cell count per padded array. Cell counts with small prime factors transform fastest.
- **Switch.** `demag = false` in `[grid]` removes the term, e.g. for Stoner–Wohlfarth comparisons (D36).

4.4 Periodic boundaries

`pbc = [Px, Py, Pz]` makes an axis periodic, with P copies of the tile on each side, as in mumax3's `SetPBC` (D45). The exchange neighbour wraps around, and Voronoi grains continue across the boundary. For demag, a periodic axis is not padded, so the FFT's own wrap-around is used on purpose, and each tensor entry sums the copies in a window centred on each cell: every cell sees $(2P+1)$ tile lengths of material, with the two end layers at half weight when n is even. The copies are a *finite* block on purpose: in 2D and 3D, the field of an infinite lattice of copies depends on the shape of the body they fill, so the shape is made explicit. Each copy uses the same Newell and dipole kernels. The sum costs about 1 s for $128\times128\times1$ cells at $P = 10$ and about

20 s for 64^3 cells at $P = 5$, once per run. The error left by a finite P shrinks slowly: halving with each doubling of P for a film, and falling 4× for a rod.

4.5 Time integration

- **`rk45`** (default at $T = 0$). An adaptive Dormand–Prince 5(4) integrator (D18). The error is the largest per-cell $|\mathbf{m}_5 - \mathbf{m}_4|$, so a few fast-switching cells in a large grid are not averaged away. A step is rejected when the error exceeds `tol`; the next step is $h \cdot 0.9(\mathrm{tol}/\mathrm{err})^{1/5}$, clamped to 0.2–5×. $\mathbf{m}$ is renormalised after every accepted step. Six field evaluations per step.
- **`heun`** (default at $T > 0$). A fixed step `dt`. One noise sample per step, held over the predictor and the corrector, which converges to Brown's (Stratonovich) equation (D42). Two field evaluations per step.
- **`rk23`**. Adaptive Bogacki–Shampine 3(2) with the noise frozen over each step. When a step is rejected, its random increment is *not* thrown away (that would favour steps where the noise happened to be small and bias the temperature); an exact Brownian bridge splits it between the shorter retry and the rest. Three field evaluations per step.

The random numbers come from a counter-based Philox4x32-10 generator, written once and shared by the CPU and GPU, keyed by the stage's `seed` and its position in the card. The same card and seed give the same run on either device.

4.6 Energy minimisation and hysteresis

`minimize` is projected steepest descent with Barzilai–Borwein step sizes (eqs. 22–23, Exl *et al.* 2014; essentially mumax3's minimiser) (D21). The direction is $-\mathbf{m}\times(\mathbf{m}\times\mathbf{H})$, the update is renormalised, and the step alternates the two BB formulas with no line search. The energy may rise on single iterations; convergence is judged by the torque only. If `max_iter` is reached the stage fails, so a half-converged state near a switching field is never reported silently.

A `hysteresis` stage minimises at each field of a sweep. With `refine_to`, a coarse step that crosses $\mathbf{m}\cdot\hat{\mathbf{h}} = 0$ or makes $\langle\mathbf{m}\rangle$ jump is undone and halved, recursively, down to `refine_to` (D31). A loop then costs about 45 field steps instead of about 600, with the switch still resolved finely.

4.7 Energy barriers

A `gneb` stage finds the minimum energy path between a state and its reverse, by geodesic NEB with a climbing image (Bessarab *et al.* 2015) (D48). A chain of images feels the true force across the path and a spring along it; the highest image climbs to the saddle, so the barrier needs no interpolation. Distances are Ms-weighted rms great-circle angles. The first path is a great circle with 10^{-2} rad of noise per cell, since a perfectly uniform path can't leave itself. L-BFGS moves the whole chain with no line search, as the forces are not the gradient of any energy (Ivanov *et al.* 2020) (D49). Each step rotates the cells, and no cell may turn more than 0.1 rad: an rms cap lets a small nucleus turn as far as it likes. While the path's energy range exceeds 10 barriers, its far end moves in to the first image lower than the start, because a pass of 25 k_BT on a slope of 10^5 k_BT would fit between two images.

4.8 Backends

The drivers (step control, BB steps, refinement) are written once. The per-cell arithmetic sits behind a small backend interface with two implementations: the CPU backend, which is the reference "oracle", and the CUDA backend, one thread per entry, with cuFFT and CUB (D24). Every CUDA kernel is tested against the CPU. Within a call, $\mathbf{m}$ stays on the device; it is copied to and from the host only at the start and end of a `run_until` or `minimize`. There is no global state, so several problems can coexist in one Python process.

5 Kinds of simulations

A card has a few setup sections, then an ordered list of stages. The magnetisation carries over from one stage to the next; the applied field does *not*: every stage states its own field, defaulting to zero. Table 2 maps common questions to stage types.

A full hysteresis loop is two `hysteresis` stages, one sweeping down and one back up. **A field slightly off**

Table 2: Typical simulations and how to set them up.

Question	Stage(s)	Notes and example
What is the ground state or a metastable state?	`minimize`	Fastest route to equilibrium. Start from several initial states and compare energies (as in μMAG SP3, flower vs vortex).
How does **m** move in time?	`llg-dynamics` with `duration`	Physical α and γ; sample with `sample_every`. Example: `examples/sp4.toml` (switching a permalloy film).
Relax with real dynamics	`llg-dynamics` with `until_torque`	Large α (e.g. 1); stops when the torque is small. Used for SP4's S-state.
Coercivity, remanence, switching field	`hysteresis`	Quasi-static sweep with `refine_to`; summary `Hc_T`, `switch_T`, `remanence`. Examples: `sp2.toml`, `cube_core_shell.toml`.
Microstructure effects	`hysteresis` + `[grains]`, `[[region]]`, `[[coupling]]`	Grains, boundary phases, soft shells, weak interfaces. Example: `grains.toml`.
Thermal fluctuations, superparamagnetism	`llg-dynamics` with `temperature`	`heun` or `rk23`; noisy time series of $\langle \mathbf{m} \rangle$. Barriers of a few $k_B T$ only (Section 8).
Energy barriers, thermal coercivity	`gneb`	The barrier against field, and where it crosses 25 $k_B T$. Reaches barriers no thermal run can. Check with a second `seed`.
Bulk or extended material	`[grid] pbc = [P, P, P]`	A periodic tile without an artificial outer surface; check the result at two values of P.
Continue from an earlier result	`[initial] file = ...`	Restart from any saved state, even on a different cell size.

the easy axis (e.g. 1°) is the convention for hard-magnet cubes: exactly along the axis, the reversed state is an exact but unstable equilibrium that a minimiser may never leave.

Example: μMAG standard problem 4, relax then switch (`examples/sp4.toml`):

```
[grid]
n    = [128, 32, 1]                    # cells along x, y, z
cell = [3.90625e-9, 3.90625e-9, 3e-9]  # m

[material.permalloy]
Ms = 8e5        # A/m
A  = 1.3e-11    # J/m

[initial]
m = [1, 1, 1]   # normalised on load

[[stage]]       # the S-state: damped dynamics until the torque is small
type = "llg-dynamics"
alpha = 1.0
gamma = 1.759457895880903e11   # rad/(T s), as muMAG specifies
tol = 1e-9                     # well below until_torque
until_torque = 1e-6
check_every = 50e-12
max_duration = 20e-9

[[stage]]       # the reversal, sampled every 1 ps
type = "llg-dynamics"
alpha = 0.02
gamma = 1.759457895880903e11
mu0_H = [-24.6e-3, 4.3e-3, 0.0]   # T
duration = 1e-9
sample_every = 1e-12
```

Example: a polycrystal (`examples/grains.toml`, abridged): eight $Nd_2Fe_{14}B$ grains with axes within 15° of z, coated by a 4 nm soft phase, swept to −6 T.

```
[grid]
n    = [20, 20, 20]
cell = [2e-9, 2e-9, 2e-9]
```

```
[material.grain]            # listed first: region 0, the background
Ms = 1281197.29             # A/m (mu0 Ms = 1.61 T)
A  = 8e-12
K  = 4.9e6

[material.boundary]
Ms = 379584.54              # A/m (mu0 Ms = 0.477 T)
A  = 6.12e-12

[grains]
count    = 8
first_id = 1
material = "grain"
seed     = 1
axes     = { mean = [0, 0, 1], cone_deg = 15 }
boundary = { thickness = 4e-9, material = "boundary", id = 100 }

[initial]
m = [0.0174524, 0.0, 0.9998477]

[[stage]]
type       = "hysteresis"
direction  = [0.0174524, 0.0, 0.9998477]   # 1 degree off z
mu0_H_from = 0.0
mu0_H_to   = -6.0
mu0_H_step = 0.1
refine_to  = 1e-3
```

6 Input card reference

Validation is strict and happens before anything runs: an unknown key, a missing key or a wrong type is an error, and every problem in the card is reported together. "req." marks a required key.

6.1 Top level and `[grid]`

Key	Type	Default	Meaning
`device`	string	`"cpu"`	`"cpu"` or `"cuda"`; `-device` on the command line overrides it
`n`	3 integers ≥ 1	req.	cells along x, y, z
`cell`	3 numbers > 0	req.	cell size (m)
`demag`	bool	`true`	include the demagnetising field
`pbc`	3 integers ≥ 0	`[0,0,0]`	periodic copies per side along x, y, z; 0 is a free axis

6.2 `[material.<name>]`

One or more named materials. The *first* one listed fills the whole grid as region 0, the background. The name `vacuum` is built in ($M_s = 0$) and reserved.

Key	Type	Default	Meaning
`Ms`	number ≥ 0	req.	saturation magnetisation (A/m); 0 is vacuum
`A`	number ≥ 0	req.	exchange stiffness (J/m)
`K`	number	0	uniaxial anisotropy (J/m^3); energy $-K(\mathbf{m} \cdot \hat{\mathbf{k}})^2$
`axis`	3 numbers	`[0,0,1]`	easy axis $\hat{\mathbf{k}}$, normalised on load

6.3 `[[region]]`: painted shapes

Regions are painted in card order, and later paints win. A cell belongs to a shape if its *centre* is inside it. The region map is `uint8`, so ids run from 0 to 255.

Key	Type	Default	Meaning
`id`	integer 1–255	req.	region id (0 is the background)
`material`	string	req.	a defined material or `"vacuum"`
`shape`	string	req.	`all`, `cuboid`, `cylinder` or `ellipsoid`
`inverse`	bool	`false`	paint everything *outside* the shape
`axis`	3 numbers	material's	override the easy axis in this region
Shape keys			
`cuboid`	`corners = [[x0,y0,z0],[x1,y1,z1]]`; lower bounds inside, upper outside		
`cylinder`	`center`, `radius`, `height`; axis along z		
`ellipsoid`	`center`, `semi_axes = [a,b,c]`		

The results record each region's exact volume next to its painted volume, and warn when they differ by more than 5%, which catches shapes clipped by the grid or too coarsely resolved. To make a sphere, paint `vacuum` with an `ellipsoid` and `inverse = true`.

6.4 `[grains]`: Voronoi polycrystals

Optional, applied after all `[[region]]` painting. It fills one region with 3D Voronoi grains: each cell joins the grain of its nearest seed. Seeds are drawn uniformly over the box, so the grains do not depend on the cell size.

Key	Type	Default	Meaning
`count`	integer ≥ 1	req.	number of grains
`first_id`	integer 1–255	req.	grains get ids `first_id` ... `first_id+count-1` (≤ 255)
`material`	string	req.	the grains' material
`seed`	integer ≥ 0	req.	same seed, same grains and axes
`within`	region id	0	the region to fill
`axes`	see text	`"material"`	easy axis per grain
`boundary`	table	none	grain-boundary phase

`axes` is `"material"` (every grain uses the material's axis), `"random"` (isotropic), or `{ mean = [x,y,z], cone_deg = θ }` (uniform within θ of the mean, $0 < \theta \leq 90$). `boundary = { thickness = t, material = "...", id = n }` coats each grain face: a cell closer than $t/2$ to its grain's boundary joins region n, so neighbouring grains are separated by t. The distance is computed exactly from the bisecting planes between seeds. t must be at least two cells; thinner boundaries are modelled with `[[coupling]]`. On a periodic axis the grains use minimum-image distances and continue across the boundary; painted shapes do not wrap.

6.5 `[[coupling]]`: exchange between regions

Rescales the exchange across faces between two regions, as mumax3's `ext_ScaleExchange` and `ext_InterExchange` do. Exchange inside a region is unchanged; later entries win.

Key	Type	Meaning
`regions`	`[a, b]`	two region ids, or `"grains"` for every grain id; $a \neq b$. `["grains","grains"]` means every pair of distinct grains
`scale`	number ≥ 0	multiplies the harmonic-mean A across those faces
`A`	number ≥ 0	sets the face exchange (J/m) directly

Give exactly one of `scale` or `A`. `scale = 0` decouples the regions completely.

6.6 `[initial]`

Exactly one of `m = [x, y, z]` (a uniform start, normalised) or `file = "path/trajectory.h5"` (relative to the card) with an optional `state`: `"final"` (default) or `"stage<k>"`, the last snapshot of stage k. A restart needs the same cell counts `n`; the cell size may differ. The file's SHA-256 goes into the provenance.

6.7 `[[stage]]`, `type = "minimize"`

Key	Default	Meaning
`mu0_H`	`[0,0,0]`	applied field (T)
`torque_tol`	`1e-6`	stop when $\max \lvert \mathbf{m} \times \mathbf{H} \rvert / M_s$ falls below this
`max_iter`	`100000`	fail after this many iterations

6.8 `[[stage]]`, `type = "hysteresis"`

Key	Default	Meaning
`direction`	req.	field direction, normalised
`mu0_H_from`, `mu0_H_to`	req.	field range along `direction` (T), inclusive
`mu0_H_step`	req.	step (T); must divide the range into whole steps
`torque_tol`, `max_iter`	`1e-6`, `100000`	as for `minimize`
`refine_to`	none	halve any step that crosses $\mathbf{m} \cdot \hat{\mathbf{h}} = 0$ or jumps, down to this (T)
`refine_jump`	`0.1`	a change $\lvert \Delta \langle \mathbf{m} \rangle \rvert$ above this counts as a jump
`dump`	`false`	save a snapshot at every field, not just the last

The stage reports `Hc_T`, the first zero of $\mathbf{m} \cdot \hat{\mathbf{h}}$ (linearly interpolated); `switch_T`, the midpoint of the largest jump; and `remanence`, $\langle \mathbf{m} \rangle$ at zero field if the sweep passes through it.

6.9 `[[stage]]`, `type = "gneb"`

Key	Default	Meaning
`direction`	req.	field direction, normalised
`mu0_H`	req.	list of field values along `direction` (T), one path each, in order
`images`	20	images on the path, counting both ends; at least 3
`temperature`	300	temperature (K), which sets $k_B T$ in the summary
`torque_tol`	`1e-4`	stop when $\max \lvert \text{force} \rvert / M_s$ is below this on every image
`max_iter`	`100000`	give up after this many iterations (the stage fails)
`seed`	0	seeds the small noise on the first path

At each field the stage minimises the current $\mathbf{m}$ and $-\mathbf{m}$ as the two end states, then finds the path. It leaves $\mathbf{m}$ at the last field's start state. It reports `barrier_J` and `barrier_kT` per field, and `Hc_T`, where the barrier crosses 25 $k_B T$ (linearly interpolated), or `null`. A run can converge to a higher saddle than the lowest, so re-run with another `seed` and keep the lowest. A saved path may end just past the saddle.

6.10 `[[stage]]`, `type = "llg-dynamics"`

Key	Default	Meaning
`alpha`	req.	Gilbert damping (dimensionless)
`gamma`	`1.76086e11`	gyromagnetic ratio (rad/(T s))
`mu0_H`	`[0,0,0]`	applied field (T)
`method`	`rk45` ($T = 0$), `heun` ($T > 0$)	`rk45` ($T = 0$ only), `heun` (fixed step) or `rk23` (adaptive)
`tol`	`1e-6`	per-step error tolerance (`rk45`, `rk23`)
`max_step`	none	largest step (s) (`rk45`, `rk23`)
`dt`	req. for `heun`	fixed step (s); only with `heun`
`temperature`	0	temperature (K)
`seed`	req. at $T > 0$	noise seed, an integer 0 to $2^{32} - 1$
`dump_every`	none	snapshot interval (s)
Stopping rule: exactly one of		
`duration`		run time (s), with optional `sample_every` (s) for table rows
`until_torque`		stop when $\max \lvert \mathbf{m} \times \mathbf{H} \rvert / M_s$ is below this; needs `check_every` and `max_duration` (s), and `tol` $\leq$ `until_torque`/100. Refused at $T > 0$

7 Choosing numerical parameters

micromag never picks the cell size, the step or the tolerances for you. These rules of thumb come from the benchmarks.

Cell size. A cell must be smaller than the shortest length over which **m** turns. Two lengths matter: the exchange length $l_{\mathrm{ex}} = \sqrt{2A/(\mu_0 M_s^2)}$, which sets vortex and flux-closure structures, and the wall parameter $\delta_0 = \sqrt{A/K}$ (a Bloch wall is $\pi\delta_0$ wide), which matters in hard magnets. For permalloy, $l_{\mathrm{ex}} \approx 5.7\,\mathrm{nm}$; for $Nd_2Fe_{14}B$, $l_{\mathrm{ex}} \approx 2.8\,\mathrm{nm}$ and $\delta_0 \approx 1.3\,\mathrm{nm}$. The benchmarks used $0.5\,l_{\mathrm{ex}}$ for μMAG SP2, $0.7\,l_{\mathrm{ex}}$ for SP4, and 1–1.3 nm for the $Nd_2Fe_{14}B$ cubes. In the hard-magnet benchmarks, coarser meshes gave higher coercive fields, so **check any result by refining the mesh once**. Keep cells close to cubic; aspect ratios beyond 1:4 are untested for the demag tensor.

Grid size. Memory and time are dominated by the demag FFT on a grid padded to $2n$ per free axis. On a laptop CPU, 10^4–10^5 cells are practical; 10^6 cells (e.g. a 100 nm cube at 1 nm) belong on a GPU.

LLG tolerances. `tol` bounds the per-step change of **m** in any cell; 10^{-6} is fine for trajectories. To stop on torque, `tol` must sit at least 100× below `until_torque`: each step's error re-excites a torque of a few × `tol`, so the threshold would otherwise never be reached. For relaxation, a large α (≈ 1) is fastest; it is unphysical for the path but not for the end state.

Minimiser tolerance. `torque_tol = 1e-6` (in units of M_s) is the tested default. Near a switching field, convergence slows, so leave `max_iter` generous rather than loosening the tolerance.

Hysteresis steps. Use a coarse `mu0_H_step` (about 1–5% of the expected switching field) with a fine `refine_to` (about 10^{-3} T for hard magnets, 10^{-5} T for soft films). The coarse step must divide the range exactly.

Thermal step `dt` (for `heun`). It must resolve the fastest precession: about 1/20 or less of $2\pi/(\gamma_0 H_{\mathrm{max}})$, where H_{max} includes the exchange field, roughly $2A/(\mu_0 M_s \Delta x^2)$ per neighbour. It should also be well below a cell's free-diffusion time $\tau_N = (1+\alpha^2)\mu_0 M_s V/(2\alpha\gamma_0 k_B T)$; the tests use $\tau_N/200$. Check a result by halving `dt` once. Femtosecond steps are typical.

8 Thermal runs: what to expect

Set `temperature` (and `seed`) on an `llg-dynamics` stage:

```
[[stage]]
type = "llg-dynamics"
alpha = 0.1
temperature = 300     # K
seed = 1              # same card and seed: the same run, on the CPU or the GPU
dt = 1e-14            # s, the fixed heun step; check by halving it
duration = 1e-9
sample_every = 1e-12
```

- **Reach.** Steps resolve precession on femtosecond scales, so direct runs reach nanoseconds to microseconds. Thermally activated reversal over a $25\,k_B T$ barrier (about a second) is far out of reach; use barriers of a few $k_B T$.
- **Cell-size dependence.** Each cell gets an independent kick, so smaller cells carry shorter-wavelength fluctuations and the magnet looks "more thermal". No correction is applied; report the cell size with any thermal result and compare at least two.
- **Double counting.** Parameters measured at temperature, such as $M_s(300\,\mathrm{K})$, already contain the average effect of sub-cell fluctuations, and the noise adds them again. micromag does not correct for this.
- **Scope.** One temperature per stage, uniform in space. `minimize` and `hysteresis` are always at $T = 0$. Each stage keys its noise by `seed` and its position in the card, so two stages with the same seed draw different numbers.

9 The results folder

`micromag run CARD -o DIR` writes:

```
DIR/
  card.toml                  the card, copied byte for byte
  provenance.json            versions, git state, timings, status, per-stage summaries
  git.diff                   only if the checkout had uncommitted changes
  stage1_llg-dynamics.csv    one table per stage: stage<k>_<type>.csv
  trajectory.h5              magnetisation snapshots and the region map
  trajectory.xdmf            sidecar for opening trajectory.h5 in ParaView
```

Stage tables. One row per sample, flushed as written (`tail -f` works). Time restarts at 0 in each stage.

- `llg-dynamics`: `t, mu0_Hx, mu0_Hy, mu0_Hz, mx, my, mz, E_exchange, E_anisotropy, E_zeeman, E_demag, E_total, steps, rejected`
- `minimize`, `hysteresis`: the same without `t`, `steps` and `rejected`, plus `iterations`.
- `gneb`: `mu0_Hx, mu0_Hy, mu0_Hz, image, s, E_total, climbing`, one row per image per field.

$\langle \mathbf{m} \rangle$ is averaged over magnetic cells only; energies are in J, fields in T, time in s. A refined hysteresis stage revisits fields, and every field it computed is a row, in the order computed.

`trajectory.h5` (float64, gzip-compressed): `/regions` is the `uint8` region map with a JSON `materials` attribute; `/stage<k>/m` holds the snapshots, shaped `(count, 3, nz, ny, nx)`, with `/stage<k>/t` or `/stage<k>/mu0_H`; `/final/m` is the end state; `/failed/m` the last state of a failed stage. By default each stage saves its final snapshot; `dump_every` or `dump = true` saves more; a `gneb` stage saves every image of every field. Open `trajectory.xdmf` in ParaView (XDMF Reader) to view **m** and the regions over all snapshots.

`provenance.json` records the status (`running`, `completed` or `failed`, with the error), timestamps, device and GPU name, package versions, the git hash and dirty flag, the initial state, each region's exact and painted volume, and per stage its wall time, the hysteresis or `gneb` summary, or the LLG method, step counts, seed and temperature. It is rewritten after each stage, so a killed run shows how far it got.

Failures. A stage fails at `max_duration` or `max_iter`, or on a solver error. The run stops, keeps everything written so far, saves the last state in `/failed`, and exits with code 1. A bad card or a non-empty output folder exits with code 2 before anything is written.

10 The Python API

Everything a card does is available from Python, for scripting parameter studies or custom analysis. Arrays must be float64 and C-contiguous in the layout of Section 4.1; a wrong dtype or shape raises an error rather than silently copying.

```python
import numpy as np
import micromag as mm

g = mm.Grid(64, 64, 1, 2e-9, 2e-9, 2e-9)         # nx, ny, nz, dx, dy, dz; pbc=(Px, Py, Pz)
s = (g.nz, g.ny, g.nx)
Ms, A, K = np.full(s, 8e5), np.full(s, 1.3e-11), np.zeros(s)
k = np.zeros((3, *s)); k[2] = 1
prob = mm.Problem(g, Ms, A, K, k, (0, 0, 0), device="cpu")   # H_ext in A/m; or "cuda"

m = np.zeros((3, *s)); m[0] = 1
mm.Minimizer(prob).minimize(m, torque_tol=1e-6)   # m is updated in place
print(prob.energies(m))                           # {"exchange": ..., "demag": ...} in J

llg = mm.LLG(prob, alpha=0.02)                    # gamma, tol, max_step, method also settable
llg.run_until(m, 1e-9)                            # advance to t = 1 ns

hot = mm.LLG(prob, alpha=0.1, method="heun", dt=1e-14, temperature=300, seed=1)
hot.run_until(m, 2e-9)

mm.run("examples/sp4.toml", "runs/sp4")          # same as the command line
```

Note that `Problem` takes $\mathbf{H}_{\text{ext}}$ in A/m, while the card takes $\mu_0\mathbf{H}$ in T. `Problem` also accepts `regions` (a `uint8` map), `coupling` (a 256×256 pair table) and `demag=False`. `prob.H_ext` can be changed between calls; changing a material means building a new `Problem`.

11 Validation and known limits

Every numerical routine is tested against an analytic result or an independent oracle (e.g. the Bloch-wall profile, Aharoni's prism factors, a direct $O(N^2)$ demag sum, the analytic macrospin, the Boltzmann distribution), and every GPU kernel against the CPU. The comparisons with published results:

Benchmark	micromag	Reference	Diff.
μMAG SP3: flower/vortex crossover (20^3)	$L = 8.4551\,l_{\text{ex}}$	8.47 (MagTense 8.4556)	−0.18%
μMAG SP4 field 1: first $\langle m_x\rangle = 0$	138.6 ps	OOMMF: 138.4 ps	+0.14%
μMAG SP2: H_c/M_s at $d/l_{\text{ex}} = 1$	0.05704	exact limit 0.05707	−0.05%
μMAG SP2: H_c/M_s at $d/l_{\text{ex}} = 10$	0.05446	OOMMF 1.1: 0.05454	−0.15%
μMAG SP2: H_c/M_s at $d/l_{\text{ex}} = 20$	0.04960	OOMMF 1.1: 0.04991	−0.6%
μMAG SP2: H_c/M_s at $d/l_{\text{ex}} = 30$	0.04533	OOMMF 1.1: 0.04594	−1.3% (open)
Stoner–Wohlfarth astroid, ψ = 15–75°		analytic	$\le 5\times10^{-4}H_K$
Periodic perpendicular film: switching at 30°	−0.79446 T	SW with $H_{K,\text{eff}}$: −0.79443 T	2×10^{-5} T
40 nm $Nd_2Fe_{14}B$ cube: switching field	5.275 T	Fischbacher 2017: 5.29 T	−0.3%
40 nm cube: 25 k_BT crossing at 300 K	3.841 T	Fischbacher 2017: 3.94 T	−2.5%
25 nm core–shell cube: H_c	2.721 T	Fischbacher 2018: 2.76 T	−0.04 T
255 decoupled random grains: M_r, H_c	$0.4705\,M_s$, $0.4627\,H_K$	SW, same axes	$< 10^{-7}$

The SP2 case at $d/l_{\text{ex}} = 30$ is a known miss, probably due to the mesh ($0.5\,l_{\text{ex}}$ cells against OOMMF's 0.60); a cell-size study is open.

Limits to keep in mind.

- Uniaxial anisotropy only; no DMI, magnetoelastics or cubic anisotropy.
- Periodic boundaries use a finite number of copies; the result depends on P and should be checked at two values.
- The applied field and the temperature are uniform and constant within a stage; α and γ are one value per stage, not per cell.
- At most 256 regions (ids 0–255), which caps a polycrystal at about 255 grains.
- One GPU per run.
- Thermal results depend on the cell size, with no renormalisation (Section 8).

Further reading in the repository. `docs/cards.md` (card keys), `docs/results.md` (outputs), `docs/thermal.md`, `docs/gpu.md`, `docs/verification.md`, `docs/architecture.md`; `DECISIONS.md` for the reasoning behind every choice, and `BENCHMARKS.md` for the reference data and sources.

References

[1] J. Fischbacher *et al.*, "Micromagnetics of rare-earth efficient permanent magnets", *J. Phys. D: Appl. Phys.* **51**, 193002 (2018). doi:10.1088/1361-6463/aab7d1

[2] J. Fischbacher *et al.*, *Appl. Phys. Lett.* **111**, 072404 (2017).

[3] A. J. Newell, W. Williams and D. J. Dunlop, *J. Geophys. Res.* **98**, 9551 (1993).

[4] L. Exl *et al.*, "LaBonte's method revisited", *J. Appl. Phys.* **115**, 17D118 (2014).

[5] W. F. Brown, "Thermal fluctuations of a single-domain particle", *Phys. Rev.* **130**, 1677 (1963).

[6] C. Rackauckas and Q. Nie, "Adaptive methods for stochastic differential equations via natural embeddings and rejection sampling with memory", *Discrete Contin. Dyn. Syst. B* **22**, 2731 (2017).

[7] J. K. Salmon *et al.*, "Parallel random numbers: as easy as 1, 2, 3", SC'11 (2011).

[8] μMAG standard problems, `https://www.ctcms.nist.gov/~rdm/mumag.org.html`.